

Library B Guide

This document should include everything you need to use this library to complete the task. You are welcome to review it as needed.

How Interactive Formulas are Implemented

Step 1: Define a formula config

Create a configuration object that defines the interactive formula. It contains the following parts - `formulas`, `variables`, and `semantics`.

```
const config: Config = {
  formulas: [...],
  variables: {...},
  semantics: function ({ vars }) {...},
};
```

Step 2: Write the formula

Write the formula in LaTeX as an entry in the `formulas` array. Assign the entry a unique `id`.

```
formulas: [
  {
    id: "linear-formula",
    latex: "y = x + 1",
  },
],
```

Step 3: Make variables interactive

Specify which variables in the formula should be made interactive and how. The variable key must **exactly match** the corresponding LaTeX token (case-sensitive). Symbols in the LaTeX that are not listed in `variables` render as plain, non-interactive notation.

```
variables: {
  y: {
    name: "Y Value",
  },
  x: {
    input: "drag",
    default: 3,
    range: [-10, 10],
    step: 0.5,
    name: "X Value",
  },
},
```

Step 4: Define the formula `semantics`

Define how input variables map to output variables in the `semantics` function. Access input and output variables as properties of the `vars` object.

```
semantics: function ({ vars }) {
  vars.y = vars.x + 1;
},
```

You can read this as computing `y` (`vars.y`) by getting the value of `x` (`vars.x`) and adding one. Refer to variables by their keys from the `variables` object.

Step 5: Render within a `Provider` and `Formula`

Render the formula by specifying a `Formula`, giving it the `id` specified in the `formulas` part of the config, and then wrapping it all in a `Provider` that is given access to the config.

```
<Provider config={config}>
    <Formula id="linear-formula" style={{ height: "300px", wi
dth: "700px" }}/>
</Provider>
```

Example Implementation (`y = x + 1`)

Here is a full example implementation of an interactive formula for `y = x + 1`, where `x` and `y` are both labeled, and changes to `x` lead to computing a derived value of `y`.

```
import { Formula, Provider, type Config } from "math-notatio
n";

const config: Config = {
  formulas: [
    {
      id: "linear-formula",
      latex: "y = x + 1",
    },
  ],
  variables: {
    y: {
      name: "Y Value",
    },
    x: {
      input: "drag",
      default: 3,
      range: [-10, 10],
      step: 0.5,
      name: "X Value",
    },
  },
  semantics: function ({ vars }) {
    vars.y = vars.x + 1;
  },
```

```
};

function App() {
  return (
    <Provider config={config}>
      <Formula id="linear-formula" style={{ height: "300px",
width: "700px" }} />
    </Provider>
  );
}
export default App;
```

Further Reference

The usage of Library B consists of three main parts:

- `formulas` – the LaTeX equations to display
- `variables` – the variables and their properties
- `semantics` – a function that defines how variables relate to each other

The **variable key** is the key you use in the `variables` object. It is the single thread that connects the **LaTeX formula**, the **variables config**, and the **semantics function**.

Using the equation `y = x + 1` as an example:

Layer	How the key `"x"` is used
LaTeX	`x` appears as a symbol in `"y = x + 1"`
Variables config	`x: { name: "X Value", input: "drag", ... }` defines its properties
Semantics function	`vars.x` reads its current value

In the `variables` configuration. For each variable, they can have the following props:

Property	Description	Example	Required?
`input`	How the user interacts with it	`input: "drag"`	For interactive variables

Property	Description	Example	Required?
`default`	The starting value	`default: 5`	For interactive variables
`range`	Min and max values for input variables	`range: [0, 100]`	For interactive variables
`step`	Increment size when dragging	`step: 0.1`	For interactive variables
`name`	Explanatory label displayed to users	`name: "X Value"`	No